

Agentic Software Development: Take the Director's Chair

Markus M. Geipel

 mgeipel.github.io  linkedin.com/in/markus-geipel



August 3, 2026

The term “vibe coding” has drawn criticism for ignoring the intellectual work that is needed to create good software. I believe that the term “coding agent” is equally unfortunate, because it reduces the work of software development to just coding. Creating good software is much more than just generating code. Coding is embedded in processes that make sure implementations fit specification, that the software is unit-, integration- and end-to-end-tested, that it is versioned, documented, deployed, published, updated, bug fixed, refactored, migrated and maintained. And despite being called just coding agents, these agents can automate vast parts of the entire process. In the remainder of this article I will refer to them just as agents for simplicity's sake, well knowing that they really are a system of agents in a harness.

When working with agents you need to learn to **let go and delegate**. Delegate all you can. Not just the coding! Sit in the director's chair and give the agents space to act. In this article I describe techniques that worked for me. I am using Claude Code but all agents build on the same principles. I am sure that you will be able to adapt the techniques to your projects and I am equally sure that you will find ways to improve them. This guide is not complete, it is a starting point. It is still day one for agentic coding.

One word of caution, though: If you work on safety-critical, regulated, or security-sensitive code you should still review and understand it line by line.

1 Levers of Automation

Even in a small, one-person project, coding is embedded in a lot of support tasks and workflows. It has always been good practice to automate all that can be automated. Let us take a quick look at

“It's unfortunate that that's called vibe coding [...] It's misleading a lot of people into thinking, just go with the vibes, you know—accept this, reject that. In reality, coding with AI is a deeply intellectual exercise.”

— Andrew Ng

<https://www.businessinsider.com/andrew-ng-vibe-coding-unfortunate-term-exhausting-job-2025-6>

A film director [...] is a person who controls a film's artistic and dramatic aspects and visualizes the screenplay (or script) while guiding the film crew and actors in the fulfillment of that vision.

— Wikipedia (retrieved June 2026)

https://en.wikipedia.org/wiki/Film_director

* This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. <https://creativecommons.org/licenses/by-nc-nd/4.0/>

your levers: the pre-agentic scripts and hooks, as well as the skills and system prompts of agentic programming.

1.1 Scripts

Scripts are the most basic way of automating tasks. If you find yourself typing the same three commands over and over again, put them in a script. The toolbox of `grep`, `find`, `sort`, `sed`, `curl`, etc. lets you automate an amazing variety of tasks. Why are we not using it more? Probably because it is hard to remember all the options and the syntax of all these tools, not to mention flow control in `bash`. Here agentic coding comes to the rescue. Agents are amazingly adept creators of `bash` (or `Powershell`) scripts. Use this power to automate. They also make it easy to give the script a well-designed and documented command line interface. With this I mean proper argument parsing and a `-help` flag that serves as documentation.

Here is what I do: When I ask Claude to do something for me, say creating versions of an image with different resolutions, I ask Claude to create a script to perform the task. In my repos I have a `tool/` folder for these scripts. I normally do not call these myself, but ask Claude to do that. So, why go to the effort of having scripts if I'm prompting Claude anyway? Calling a script directly is reproducible and saves tokens. They can also later become integral to hooks and skills.

See that the agent builds scripts with a well-organized command interface including a `-help` option. The help makes the script self-documenting and thus easier for the agent to integrate efficiently into skills or its normal operation later.

1.2 Hooks

Many programming tools and systems offer hooks that can be used to add automation in the form of scripts being called automatically. Git offers hooks such as a pre-push hook, that lets you run checks before pushing. Angular CLI has hooks, as does Firebase CLI. NPM lets you add hooks to `package.json`. Basically all build systems offer lifecycle hooks. So, instead of manually building the website before running `firebase deploy`, let Firebase do the compilation automatically via a predeploy hook.

Also IDEs offer hooks: In VSCode you can, among many other things, hook into the file save command. That means you can automatically perform an action on the file, such as pretty printing, every time it is saved. This can be very useful to enforce style guidelines.

Finally, the agent itself might offer hooks. Claude Code for instance offers hooks that can be attached to tool calls. Here is an example: You need code to be always free of linter errors when deploying it to the cloud. You could ask Claude to respect the linter rules. But that is not a guarantee and also inefficient. So for the write file tool you attach a hook that runs the linter automatically

whenever the file is a source file. If there are linter errors, Claude will be instantly aware of them and fix them.

Again, it is hard to remember all the available hooks and lifecycle models of all frameworks, IDEs and agents, but agents know them all. Let them automate for you.

1.3 Agent Skills

Skills are a way of adding procedural knowledge to agents. In most agentic coding systems they are just markdown files that explain how something is done, with optional scripts and reference material. Agents can activate skills on their own, but what makes skills an automation device is that they can be explicitly called by you. In Claude Code, for instance, this is done with the forward slash and autocomplete. It is blazingly fast to call a skill. So, if you prompt the agent regularly to do the same thing, save the effort of typing and create a skill instead. Or better, let the agent create the skill for you! We will see examples later.

Do not write skills yourself! Let the agent develop the skill, let the agent run it, and instead of analyzing the result yourself, let the agent do an assessment and suggest improvements.

1.4 Agent System Prompt

All agents offer a way to add general instructions to their system prompt. Claude uses the CLAUDE.md, which is always loaded into context whenever Claude works in the folder that contains it, or in any of that folder's subfolders. That is a convenient way to automate procedures and scope the automation to particular parts of the project. For example: Instead of prompting the agent to implement A and then to update the README accordingly, add a rule to the context that says: Every time you add a feature that changes the interface of the software, update the README. Now you can just prompt: "Implement A", and the agent will automatically take care of the README. Shorter, faster, and you won't forget the README update!

2 Automate the Code-Compile-Test Cycle

In classical coding, we write code, then compile it (for scripting skip this), and then we run it and see if it is what we wanted. If not we analyze the gap and repeat the cycle until we are happy with the result. Ideally this is automated by a test. If we are not happy, the cycle repeats. Agents follow the same pattern.

To make this as efficient as possible, we want to extract ourselves as much as possible from this cycle. In other words the agent should cycle as efficiently as possible without coming back to us. I am writing explicitly "as efficiently as possible" and not "as long or often as possible"! Too many cycles often lead down a rabbit hole. Our goal is a few efficient, high-achieving cycles. There are at least three measures we can take to grease the cycle:

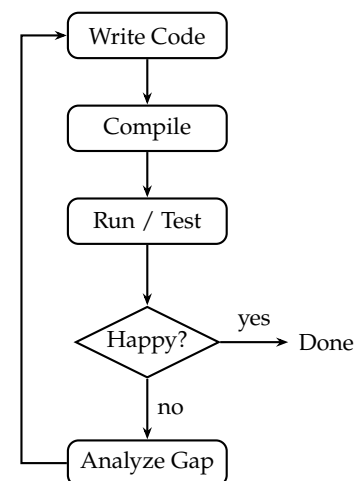


Figure 1: The code-compile-test loop. The goal is to make the loop as autonomous as possible.

2.1 Comprehensive Specification

Open questions break the cycle. This is why it is important to have them answered before the agent starts coding. The way to do this is to have a comprehensive specification of the feature or refactoring you want the agent to implement. That sounds like work, but we can automate this also. More on specification automation in Section 5.

2.2 Automate Information Access

Without the right, up-to-date information, agents can go down a rabbit hole. An unproductive loop is a waste of time, tokens and money. A way to avoid this is to make sure that all necessary information is available.

For instance, if the implementation is supposed to use a framework such as Angular, we can give the agent skills that provide Angular best practice and the latest coding guidelines. This is important for fast-moving frameworks as the information trained into the agent is most likely outdated. The makers of Angular provide these skills and we just have to install them. Many other frameworks do the same. For example Genkit, Firebase, etc. If there are no ready-to-use skills, we can also give the documentation URL directly to the agent. It will fetch the information on its own.

2.3 Automate Means of Testing and Investigation

To iterate, the agent needs to know what was achieved and what is still missing. You can take a look and give feedback, but you are blocking the cycle. We want the agent to run as many iterations autonomously as possible and that is enabled by automated testing. Unit testing is the minimum. Test aggressively—agents are really good at writing unit tests. If your project is a bit more complex, think about integration testing, system testing, UI testing, whatever it takes to check that what was implemented is correct.

Give the agent the concrete means to test and investigate. Depending on what you build that can be different things. Here are a few examples:

Web frontend Give the agent a way to run the webpage and look at and operate a browser (see Playwright for instance).

REST server Make sure the agent has `curl` and knows it. If the server needs authentication, give the agent a dev API token that only works for the local dev build.

Performance-critical software Let the agent set up performance tests and make sure it has a profiler and knows it.

Software calling external services Let the agent set up a local emulator or a mockup for the external services that at least logs requests, so that the agent can verify correct calling patterns.

Example: This article uses the KaohandT class for $\text{\LaTeX}$. However, there is no skill for Kao. No problem, let your agent create one. Just point it to the Kao documentation and examples on github (<https://github.com/fmarotta/kaobook/>). Apply the same method to any other framework or library that does not come with a skill.

Example: When working with $\text{\LaTeX}$, the cycle is broken: The agent can create $\text{\LaTeX}$, check it compiles, but cannot check if the layout and figures look good. So give your agent a tool to convert pdf to png, the agent can inspect. For instance the `pdftoppm` command. If you're thinking you would not have known about the `pdftoppm` command—neither did I. I let the agent figure it out and set it up.

Software calling a database Have a dev database ready and give the agent the CLI tools to inspect it. Docker makes it particularly easy to spin up a throw-away database.

Some agents already ship with extended testing capabilities; some don't. If something is missing, ask your agent to set it up based on your requirements. In short, make sure that the agent has everything it needs to determine success or failure, and let the agent install all investigative tools needed: profilers, debuggers, logging, telemetry, database CLIs etc.

3 Automate your Version and Release Bureaucracy

Proper versioning is incredibly important, difficult and boring. Committing code changes might seem trivial. Hit commit and type a message. Have a closer look and you notice it is anything but trivial: The messages should be consistent and concise. One commit should bundle a self-contained set of related changes. You also want to make sure not to commit any secret keys, gigantic files, or temporary artefacts. If there is a feature or issue tracking system, the commit message should reference the feature or issue that triggered the implementation. You might want to work with branches and tags as well. That covers only the source-code side of versioning. Versioning releases with semantic versions is a whole other story. I think you get my point: Versioning and releasing done right is a complicated process. One that you definitely want to automate.

The first and most important skill to create is the `/commit` skill. Again, do not write it yourself, let the agent do it. The skill should cover the entire commit process, and rules for it.

Next, if your software is released as versioned packages (npm packages, python wheels, etc.) create a `/release` skill. This skill takes care of increasing the version number in all the package manifests according to whatever logic you use to assign version numbers. I highly recommend semantic versioning (see <https://semver.org/>) by the way. The next steps in the skill depend on how your release process works. A common way is to tag the commit and let CI do the publishing. In that case you would instruct the agent to tag and push in the skill. Add additional sanity checks and steps, depending on your process. Do you have more than one package, all of which are supposed to be versioned in lock step? Should there be release notes, summarizing all the changes since the last version? Do you need a social media message to announce the new release? You get my point.

Likewise, if your software is deployed to a server or cloud infrastructure, a `/deploy` skill might be useful. Again, pin down whatever you have as deployment process and add sensible pre and post deploy checks and rules.

Example: My `/commit` skill normally covers the following process:

1. Get the current `git status`
2. Decide what to stage. Never stage temporary files or secrets! Extend `.gitignore` if necessary
3. Write commit message according to convention
4. Ask user to confirm
5. `git commit`
6. Verify with `git status`

With these skills in place, “Release early, release often” has never been faster and easier.

4 Automate Best Practice

There are many best practices in software development; we know them, but we do not always follow them because we forget or we lack the time or discipline. Agents do not tire of routine work and, correctly instructed, do not forget or take shortcuts. So besides all the coding guidelines and conventions you might want to pin down in the agent’s system prompt (e.g. `CLAUDE.md` for Claude Code), you should add best practice procedures. Here a few examples of best practices that can be automated.

4.1 Regression Tests

Every time a bug is fixed, we want to make sure that it does not reappear in the future. So with every bug fix there should be a regression test that checks for the bug. We do not want it to reappear in the future and force us into a game of whack-a-mole. Ideally, we follow test driven development and write the regression test first, then implement the fix.

When you simply ask the agent to fix a bug, it (at least at the time of writing) will dive directly into the implementation. A good place to automate regression test writing is in the system prompt of the agent.

While we are at it, we can also reinforce test writing in general. Each new feature should be tested for all corner cases. Instructing the agent to reason explicitly about corner cases results in more comprehensive tests. For each refactoring, the agent should start by pinning down the current behavior of the system and write tests that check for it before refactoring.

4.2 Code Reviews, Debugging and More

There are many more best practices that can be automated and there is a general agreement what is good practice. This means that chances are good that you do not have to reinvent the wheel, but can find a skill that implements the best practice you want to enforce.

At the time of writing the “Super Powers” skill set is one of the most popular sets of best practice skills. It is available on GitHub at <https://github.com/obra/superpowers> for a wide range of agents. In particular you find skills for code reviews, debugging, refactoring, and more. Just make sure not to blindly install skills and keep your skill set limited to skills that are relevant to your project. More on that in Subsection 7.3.

5 Automate your Specs

Good specifications are the basis of good implementations. Done right, the specifications let the agent see the bigger picture and help it to create a coherent software architecture instead of an incoherent pile of code. While specifications invoke the horror of lifeless bureaucratic paperwork, with agentic automation they can become an easy to maintain, up-to-date knowledge base.

I will work with a minimal setup of markdown files in a specs/ folder. See Figure 2 for an example layout. Depending on the size of the project you may have many more processes or you need to create and manage specs in dedicated systems outside your code repository. I am confident though, that you can transfer and scale the methods described here to bigger projects.

5.1 Create a Living Spec

Having living and up-to-date specs and documentation is the noble ambition of every project. The problem is that updating and evolving specs is for most humans as juicy as eating chalk. Apart from that, it is hard to keep a large changing spec internally consistent. Agents on the other hand really excel here. Use this!

Put all the change logic into the agent system prompt. Ideally scoped to the spec folder as shown in the example: For Claude we put a `CLAUDE.md` in the spec folder. In it we tell the agent how work with the spec. For instance:

1. After the implementation of a feature or refactoring, move it from `planned/` to `done/`
2. If the implementation had to deviate from the spec for any reason, correct the spec such that it is in line with the implementation
3. If a new feature revises parts of older features, add comments and references to both.
4. When creating new features, check for consistency with existing ones and make the user aware of potential conflicts.
5. ...

There is much more you can add: logic to reference between all the artifacts in the spec, for instance. We take a closer look on cross-referencing in Subsection 5.3.

Finally, how are the artifacts in the specs created? Obviously by the agent, based on our input! That works already well as we have rules in `CLAUDE.md` (or the equivalent file for your favourite agent), as well as the templates which give already ample guidance. We can go further and also structure the creation process with skills for each artefact type. For instance a `/create-feature` or `/create-usecase` skill. The choice is yours.

5.2 Build Intelligent Specs

Smart specifications give you a reason why things are specified the way they are. They give you a rationale. That is important both

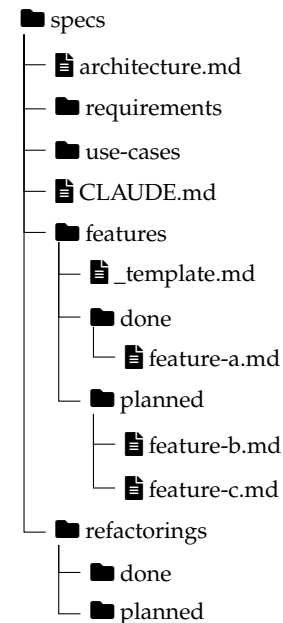


Figure 2: A minimal specs folder layout.

for humans and agents. If the specifications omit this information, they cannot evolve. Here is a simple schema to achieve this:

What should be achieved. For instance, by this feature, user story, etc.? That is the problem domain view. If you have dedicated requirements engineering you create references here, so the “what” is motivated by requirements.

How is the feature supposed to be implemented? That is the solution domain.

Why do you want to achieve it this way? This is the bridge between what you want to achieve and how you intend to achieve it.

Example: (What) Users should be able to save their preferences. (How) Use a backend database . . . (Why) because the preferences should survive clearing of the browser cache.

The beauty of this separation is that such specs can evolve and live. You can challenge the “Why”. You can change the “What” and reason what that means for the “How”. This is especially true for agents. If you give an agent a spec with only the technical solution, the agent will treat it as law. It will even tell you that the latest thing you asked it to do cannot be done because it violates the spec! If you implement the What-How-Why separation, agents will work flexibly and intelligently with the spec.

5.3 Build in Traceability

Ideally you, and the agent, can trace which requirement motivated which feature, which feature was implemented by which code, and which test is assuring its correctness. Figure 3 shows one possible link structure to implement this.

In the same way we defined processes on the specification in Subsection 5.1. We add the reference rules to the agent system prompt. In the case of Claude they go into CLAUDE.md. In the markdown files we can use the markdown reference syntax. In code files we can embed magic words in the comments. The agent will now take responsibility for creating and maintaining the references.

5.4 Discuss and Analyze your Specification

You now have a spec that evolves and keeps up-to-date automatically. That by itself is great, but you can do more. When I create a specification (with an agent) I often ask myself: How many of the requirements are already covered? Are features missing to address all requirements? Is my architecture still consistent with the requirements and features? Which features are needed by which user persona? And so forth. With a well-organized cross-linked specification, the agent can answer all these questions. Agents can give you a bird’s eye view on relationships: Ask the agent for instance to create a requirement-feature coverage matrix. Or a persona-feature usage matrix. For larger projects, these matrices can be quite insightful.

This might remind you of Simon Sinek’s golden circle of Why-What-How, but it is conceptually different. It has another purpose. Nonetheless, Simon Sinek’s golden circle is great. Have a look:

<https://simonsinek.com/golden-circle>

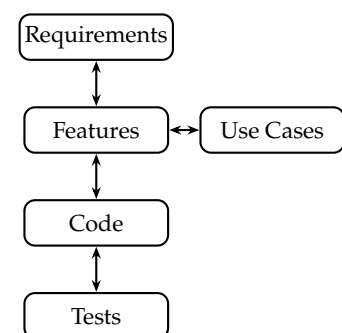


Figure 3: One possible way to structure cross-references between specification artefacts.

The spec system I used here is rather simplistic. If you want or need serious requirements engineering take a look at **Software Requirements** by Karl Wiegers, and Joy Beatty

<https://www.oreilly.com/library/view/software-requirements-3rd/9780735679658/>

6 Automate your Infrastructure

Even a small project does not live exclusively in the IDE. Or at least it should not. It should be versioned and continuously integrated and deployed. Continuous integration (CI) happens outside the IDE, normally in the cloud or on dedicated servers. So your agent in the IDE does not see it per se.

There are many more ways, in which your project may have a live outside the IDE: In case of a web application, it is deployed to a server or cloud infrastructure. Failures and other log-worthy events happen remotely and have implications for the development process. The same is true for SaaS applications, mobile applications, APIs deployed on API gateways, databases. Even desktop applications call home with telemetry and error reports.

6.1 Let your Agent Reach out

Let your agent reach out to this remote infrastructure to act on crash reports, telemetry, and other events; to investigate and debug issues; to manage and configure the infrastructure.

In many cases the infrastructure in question has APIs and CLIs that can be called from the IDE. For instance, GitHub has `gh` (<https://cli.github.com/>) and GitLab has `glab` (<https://gitlab.com/gitlab-org/cli>). Authorized with a personal access token, these CLIs can be used to manage issues, pull requests, and other project resources. We can easily create a skill which gives these capabilities to the agent. The same is true for cloud providers such as AWS, Azure, and GCP. They all have CLIs that can be used to read data from and manage the cloud infrastructure.

So creating a skill that gives the agent access to the remote infrastructure is easy: Provide the agent with the CLI and an access token, and let it create a skill that implements the desired workflows.

Example: Create a skill to check and fix the CI pipeline. First, get an API token with read access for the CI/CD system. Put it in an environment variable. Install the CLI for your CI/CD system. Then, tell the agent to create a skill based on this information and the following workflow:

1. Get list of recent builds and their status.
2. If build broken, get logs and analyze them.
3. In case of failure, try to fix it.
4. commit and push the fix
5. recheck the build status

6.2 Safety and Security

You do not want your agent to delete a production database or crash your company website, so a brief note on safety and security is in order:

1. Keep the scope of access to the minimum needed to fulfil the task. For most tasks, read-only access is sufficient.
2. Limit the resources that can be accessed: Only give access to the specific project, database, etc. that is needed.
3. Add rules to the skill that instruct the agent to report and ask before critical actions. (This is not watertight because it is just a prompt.)
4. Make the agent ask for approval before critical tool calls (via config, not prompt). The report instruction from before makes the approval decision easier for you.
5. Always have deployment stages. Minimum two: test and production. Avoid direct write access. Try to gate write access

via CI processes and grant such privileges only ever for the lowest deployment stage.

7 Final Thoughts

Here are a few miscellaneous thoughts and considerations to round off the discussion . . .

7.1 What about MCP?

I have written a lot about skills and hardly mentioned MCP. Two reasons for that: First, skills are easier to create: They are just markdown files with optional additional resources or scripts. MCP servers are software projects in their own right. Skills can use MCP or command line tools/scripts. I prefer the latter as MCP calls are tool calls for the agent. The result of tool calls is added to the context which creates a hard size constraint on the data that MCP can return. For command line tools, the agent can decide if it wants the result in its context or just piped into a file or another command line tool such as grep. This gives the agent the option to process vast amounts of data without filling up its context. So, skills with command line tools are simpler and at the same time more powerful. Having said that, MCP also has a few advantages: returning data with a documented structure, for instance. But I think that the command line still wins.

A word of caution: With the great power of the command line comes great responsibility! One way to mitigate the risk is to always work in a devcontainer. So, that is yet another good reason to have a devcontainer!

Seriously, you should always work in a devcontainer! "Do you work in a devcontainer?" should be number 13 on the Joel Test.

<https://www.joelonsoftware.com/2000/08/09/the-joel-test-12-steps-to-better-code/>

7.2 Make or Copy?

Throughout this article I have pushed towards letting the agent write bespoke skills for your particular project. Even though making your own tool or skill has never been easier, you should not constantly reinvent the wheel. Especially for specific frameworks I would always use the skills the creators provide. Also for more general tasks, there are good skills out there waiting to be reused: Take a look at <https://github.com/obra/superpowers> for instance.

Just a word of caution: You can easily get lost in the stream of ever new skills that are hyped as the next mind-blowing game changer. It is time-consuming to seriously evaluate all of them. Focus and note that most agents already come with a built-in set of well-curated skills.

Where to find skills and how to manage them? Most agents have their marketplace and plugin mechanism. However, many creators of skills want to offer agent agnostic skills. Projects such as `skills.sh` (<https://www.skills.sh/>) offer command line tools that manage your agentic skill set for a large variety of agents.

7.3 Less is More

You might be tempted to expand your agent with all the skills you can find. Intuitively, more skills seem better, right? Not really! Keep in mind: every skill consumes a bit of the agent's context. And every additional skill makes it harder for the agent to judiciously

choose the right skill for the right situation. So, limit the set of skills to the ones that your project really needs instead of drowning your agents in a maximal skill set.

7.4 Escape Hatches

Agents are trained to follow instructions. This is a desirable trait, but can also be limiting. For instance if we define a template for feature specs, the agent will follow it, even if it is not a fit. It will often bend over backwards instead of deciding to break the format for a good reason.

You can counteract this by building in escape hatches. In a template for a feature spec you can include something like a “Miscellaneous” section that is open and gives a place for all the points the agent could not fit in the rest of the structure. It is an escape hatch for extra points the agent thought of and you did not.

You can apply the same in prompts: Instead of issuing commands that the agent will follow without questioning, choose an open formulation. Tell the agent what you want to achieve instead of what to do exactly. When you have ideas about the implementation details, formulate them as suggestions to nudge the agent instead of commanding it. I also tend to finish prompts with an escape hatch such as “What do you think?” or “Did I miss something?”. This gives the agent the chance to participate. It often surfaces constructive comments and additions that did not occur to me.

Also in the interaction with humans, you learn more by giving suggestions than by giving commands.

7.5 Respect your Agent’s Training

Do not micro manage your agent. Give it space to act and express itself. Let it decide the details on its own. For instance, module, class, variable, and function names. If the agent chooses these names, they are the ones the agent will, in future, also associate intuitively with the given context. This means that the agent probably understands code it has freely written better than code where names were dictated by someone else. Probably the same applies to project structure, config file formats, etc. Respect your agent’s pre-trained preferences. One caveat: in larger projects you should pin conventions once, explicitly, and enforce consistency (e.g. in `CLAUDE.md`). And you might have guessed it: let the agent do it.

7.6 Take the Director’s Chair

I think in agentic software development we need to learn to let go of the details. We need to learn to delegate. At the same time we need to become the designers of the automated development process, guiding the agents in the fulfillment of our vision. As programmers we move from being actors to being directors.

*This is your world
You’re the creator
Find freedom on this canvas*
— Bob Ross